

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1 - INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	G. Naveen Kumar Reddy	Reg.No.:	192525288
Department:	AIML	Date:	

ANNEXURE A INDUSTRY PROBLEM SOLVING TASK

1. Smartphone Performance Optimization

A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.

2. Cloud Server Memory Bottleneck

A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.

3. Autonomous Vehicle Data Processing

An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.

4. AI Inference Edge Device

An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.

5. High-Performance Gaming Console

A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct

I **G. Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G. Naveen Kumar Reddy

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

ANSWERS TO INDUSTRY PROBLEM SOLVING TASK

Q1. Smartphone Performance Optimization

1. Understanding of the Industry Problem

Users experience visible lag when switching between running apps. This is a classic 'context-switch stall' problem: every time the OS swaps the foreground process, the CPU pipeline stalls on cache misses and, if the working set does not fit in cache, on DRAM (main memory) accesses. The constraints of a smartphone are: limited die area for cache, a tight power/thermal budget, and DRAM (typically LPDDR) shared between the CPU, GPU and other SoC blocks. The real problem is therefore not simply 'not enough memory' but a poor match between the cache hierarchy's capacity/associativity and the access pattern created by rapid app switching (many distinct working sets competing for the same L1/L2/L3 space).

2. Application of Engineering Concepts

- L1 Cache: split I-cache/D-cache, 32–64 KB per core, 1–4 cycle latency, direct-mapped or low-associativity — fastest but tiny, so it is quickly evicted (thrashed) when the OS context-switches between apps.
- L2 Cache: 256 KB–1 MB per core cluster, ~10–15 cycle latency, higher associativity — absorbs L1 misses but is still per-core/per-cluster, so it does not help when a different cluster (e.g., 'efficiency' cores) picks up the new app.
- L3 / SLC (System-Level Cache): 2–8 MB shared across all cores and the GPU, ~30–40 cycle latency — this is the level most relevant to app-switch lag because it is shared and large enough to retain a recently-used app's working set for a short time.
- DRAM (LPDDR4X/5): 100–300 cycle (tens of ns) latency, high capacity (6–16 GB) but far slower and more energy-costly per access than any cache level — every L3 miss during an app switch pays this full penalty, which is the dominant source of the observed lag.

3. Proposed Hierarchical Memory Redesign (Solution Design)

The redesign targets the specific bottleneck — L3/SLC misses during context switches — rather than blindly enlarging every level:

- Increase Shared L3/SLC to 6–8 MB with higher associativity (8–16 way) so that the working sets of 2–3 recently used apps can co-reside without evicting each other.
- Add a victim-cache buffer between L2 and L3 to capture lines evicted during a switch, reducing the number of cold DRAM fetches when the user switches back.
- Use way-partitioning / cache QoS so the foreground app is guaranteed a minimum number of L3 ways, preventing background services from thrashing it.
- Adopt a prefetcher tuned to the app-launcher: on a predicted app switch (e.g., recent-apps swipe), prefetch the target app's code/data pages from DRAM into L3 ahead of time (speculative/next-app prefetching).
- Use LPDDR5X instead of LPDDR4X for the main DRAM to raise bandwidth (~6400+ MT/s) and lower access energy, reducing the penalty of the L3 misses that remain.

4. Use of Tools

- gem5 / ARM DS-5 / Streamline — architectural simulation and cache-hierarchy design-space exploration.
- CACTI — to estimate the area, latency and energy trade-offs of enlarging L3/SLC.

- Perfetto / Android Studio Profiler — to capture real app-switch traces (cache-miss and page-fault counters) from actual devices.

5. Analysis

Representative latency/throughput comparison used to justify the redesign:

Memory Level	Typical Size	Latency	Bandwidth	Role in App-Switch Lag
L1 Cache	32–64 KB	1–4 cycles	Very high	Thrashed instantly on switch
L2 Cache	256 KB–1 MB	10–15 cycles	High	Partially retains recent data
L3 / SLC	2–8 MB (proposed)	30–40 cycles	Medium-High	Key level to enlarge/partition
DRAM (LPDDR)	6–16 GB	100–300 cycles	Medium	Dominant lag source on miss

Expected outcome: enlarging and partitioning the L3/SLC plus predictive prefetching reduces DRAM-miss rate during a switch from a high baseline to a small fraction, cutting perceived app-switch latency while LPDDR5X improves the throughput of whatever residual DRAM traffic remains — jointly reducing both latency and improving throughput as required by the problem statement.

Q2. Cloud Server Memory Bottleneck

1. Understanding of the Industry Problem

A cloud provider's database workload shows high query latency. Database workloads are characterized by large, poorly-predictable working sets (indexes, buffer pools) that routinely exceed cache capacity, causing frequent cache misses; and by memory footprints that exceed physical RAM, forcing the OS to page data to/from secondary storage. The engineering challenge is to decide how much of the latency is a cache-hierarchy problem (data not in L1/L2/L3) versus a virtual-memory problem (data not even resident in DRAM, requiring page faults to disk/SSD), and then choose the right memory technology at each level to fix it cost-effectively at data-center scale.

2. Application of Engineering Concepts — Cache Hierarchy vs Virtual Memory Paging

Aspect	Cache Hierarchy (L1–L3)	Virtual Memory / Paging
Purpose	Bridges CPU–DRAM speed gap	Extends addressable memory beyond physical DRAM using disk/SSD
Granularity	Cache lines (64 B)	Pages (4 KB–2 MB with huge pages)
Managed by	Hardware (transparent)	OS + MMU/TLB (software-assisted)
Miss penalty	Tens of cycles (L3 miss to DRAM)	Millions of cycles (page fault to SSD/HDD)

Relevance here	Reduces average query latency for hot data	A page fault under database load is catastrophic for tail latency
----------------	--	---

Because a page fault is roughly 3–4 orders of magnitude more expensive than a cache miss, the highest-leverage fix for database-query latency is to minimize page faults first (through capacity/technology choices), and only then fine-tune the cache hierarchy.

3. Recommended Improvements (Solution Design)

- Right-size DRAM capacity so that the active buffer pool / index set fits in DRAM, eliminating routine paging to slower storage — the single biggest latency win.
- Use large/huge pages (2 MB / 1 GB) for the database buffer pool to reduce TLB misses, which otherwise add an extra memory access on every translation.
- Introduce NVDIMM/persistent memory (e.g., Intel Optane-class or STT-MRAM based) as a tier between DRAM and SSD storage for warm data, so a 'page miss' degrades to a fast persistent-memory access instead of a full disk fault.
- Increase shared L3 cache and enable NUMA-aware cache/memory placement so that each socket's queries hit local, low-latency memory rather than crossing interconnect links.
- Replace SATA/SAS SSD swap/storage with NAND-flash NVMe SSDs to shrink the page-fault penalty for any paging that remains unavoidable.

4. Use of Tools

- Linux perf / Intel VTune — to measure L1/L2/L3 miss rates and page-fault counts on the live database server.
- DRAMSim2 / Ramulator — DRAM-subsystem simulation to size capacity and channel bandwidth.
- Database-specific monitors (e.g., MySQL Performance Schema, PostgreSQL pg_stat) — to correlate query latency with buffer-pool hit ratio.

5. Analysis

Memory Technology	Latency	Bandwidth	Suitability for DB Server
SRAM (L1–L3)	1–40 cycles	Very high	Caches hot rows/index nodes
DRAM (DDR5)	~60–100 ns	High (channel-scaled)	Primary buffer-pool memory
Persistent/NVDIMM	~150–350 ns	Medium-high	Warm-tier, survives page misses cheaply
NVMe SSD (NAND)	~10–100 μ s	Medium	Cold storage / fallback swap
HDD	~5–10 ms	Low	Avoid for active paging entirely

Given the four to five order-of-magnitude latency gap between DRAM and disk-based paging, the analysis shows that eliminating page faults (via adequate DRAM sizing plus a persistent-memory tier) yields far greater query-latency reduction than any feasible cache-hierarchy tuning alone — hence the recommendation prioritizes memory-technology and capacity changes over pure cache redesign.

Q3. Autonomous Vehicle Data Processing

1. Understanding of the Industry Problem

An autonomous vehicle continuously ingests high-bandwidth sensor streams (LiDAR point clouds, camera frames, radar) and must run perception/planning algorithms within a hard real-time deadline. The memory system must simultaneously deliver very high sustained bandwidth (for streaming sensor data) and very low, predictable latency (for safety-critical control-loop decisions), while operating within an automotive power and reliability envelope (wide temperature range, functional-safety requirements).

2. Evaluation of SRAM, DRAM, and MRAM

Technology	Latency	Bandwidth	Volatility / Endurance	Best Role
SRAM	Sub-ns to a few ns	Very high, on-chip	Volatile, unlimited endurance	On-chip buffers/caches for control-loop data
DRAM (LPDDR5/GDDR6)	~50–100 ns	Very high (channel-scaled, GB/s)	Volatile, refresh overhead	Frame/point-cloud buffering, model weights
MRAM (STT-MRAM)	~10–30 ns	Medium-high	Non-volatile, high endurance	Safety-critical state storage that must survive power loss

3. Proposed Memory Hierarchy (Solution Design)

- L1/L2 SRAM caches on each compute core (CPU/GPU/NPU) for the innermost loops of perception algorithms — guarantees the lowest possible latency for control decisions.
- A large on-chip SRAM scratchpad (tightly-coupled memory) for streaming sensor tiles, so the accelerator processes data without waiting on DRAM for each tile.
- High-bandwidth DRAM (LPDDR5/GDDR6/HBM-class) as main memory to buffer full LiDAR/camera frames and hold neural-network weights, sized and channel-parallelized for the required GB/s of sustained sensor throughput.
- MRAM for non-volatile, fast-access storage of critical vehicle state (last known safe trajectory, calibration data) so it survives a power glitch without the write latency/energy cost of flash.
- A dedicated, isolated memory partition/QoS channel for the safety-critical control loop, so best-effort sensor-fusion traffic cannot starve it of bandwidth (deterministic memory access, key to real-time guarantees).

4. Use of Tools

- Simulink / ROS 2 with memory-profiling plugins — to model sensor data-flow and required bandwidth.
- gem5 / DRAMSim2 — architectural exploration of the SRAM-DRAM hierarchy under streaming workloads.
- NVIDIA Nsight Systems (for DRIVE-class SoCs) — real hardware profiling of memory bandwidth utilization during sensor-fusion execution.

5. Analysis

A modern L4 LiDAR alone can produce on the order of hundreds of MB/s to a few GB/s of point-cloud data, and multi-camera arrays add several more GB/s; control loops typically require sub-10 ms deterministic response. The proposed hierarchy meets this by using SRAM (ns-latency) for the deterministic control path and wide, parallel DRAM channels for the bulk sensor bandwidth, while MRAM removes the risk of losing safety state on power interruption — jointly maximizing bandwidth and minimizing latency as required.

Q4. AI Inference Edge Device

1. Understanding of the Industry Problem

An edge AI device (e.g., a smart camera or IoT inference module) must load trained model weights quickly after power-on or wake-from-sleep, while operating on a very limited energy budget (often battery-powered). The core tension is between NAND Flash's non-volatility and low idle power versus DRAM's speed versus RRAM's emerging balance of both — the design must choose/combine technologies so the model is available fast without draining the battery.

2. Comparison of NAND Flash, DRAM, and RRAM

Technology	Latency	Bandwidth	Power (idle/active)	Volatility
NAND Flash	~10s of μ s (read)	Low-medium	Very low idle, low active	Non-volatile
DRAM	~50–100 ns	High	Higher idle (refresh), moderate active	Volatile — loses model on power-off
RRAM	~10–50 ns	Medium	Very low idle, low active	Non-volatile, in-memory-compute capable

3. Recommended Optimized Hierarchical Structure (Solution Design)

- Store the trained model persistently in NAND Flash (or RRAM where available) so no re-download/re-training is needed and zero power is spent retaining it when the device is off.
- Use RRAM as a fast non-volatile intermediate tier: it can hold frequently-used model layers/weights and be read at near-DRAM speed with far lower idle power, cutting the 'cold start' load time compared to Flash-only designs.
- Use a small DRAM (or SRAM-heavy, DRAM-light) working memory only for active-inference intermediate activations, keeping DRAM capacity — and hence its refresh-power cost — minimal.
- Where the accelerator (NPU) supports it, exploit RRAM's in-memory compute (analog MAC) capability to perform part of the inference directly in the memory array, cutting both data movement and energy further.
- Apply weight compression/quantization so the persistent-storage footprint and the amount that must be streamed into fast memory at load time are both reduced.

4. Use of Tools

- TensorFlow Lite Micro / TVM — to quantize and package the model for constrained edge memory.
- NVSim / CACTI (with RRAM/NVM extensions) — to estimate RRAM/Flash array latency, energy and area.
- Vendor SDK power profilers (e.g., for ARM Cortex-M/NPU edge boards) — to measure real load-time and energy for each memory configuration.

5. Analysis

Loading a multi-MB model directly from NAND Flash into DRAM on every wake incurs both the Flash read latency and DRAM refresh energy for the full duration the model is resident. Caching hot weights in RRAM instead reduces effective load latency by roughly an order of magnitude versus Flash while keeping idle power close to Flash levels (since RRAM needs no refresh), giving the best combination of fast model loading and low power consumption requested in the problem.

Q5. High-Performance Gaming Console

1. Understanding of the Industry Problem

Frame drops during large scene loads occur when the GPU/CPU must stream large volumes of new texture, geometry and level data faster than the memory system can supply it, stalling the render pipeline. This is fundamentally a throughput (bandwidth) problem during burst loads, layered on top of the console's steady-state latency requirements for real-time rendering.

2. Analysis: L3 Cache vs DRAM Throughput

Level	Typical Latency	Typical Throughput/Bandwidth	Behavior During Large Scene Load
L3 Cache	~30–40 cycles	Very high, but capacity-limited (a few MB–tens of MB)	Quickly saturated/evicted — new scene data is far larger than L3 capacity
DRAM (GDDR6/GDDR6X)	~100–300 cycles	Very high sustained bandwidth (hundreds of GB/s), capacity in GBs	Becomes the real bottleneck when asset-streaming bandwidth is under-provisioned or contended with rendering traffic

Because a large scene load moves far more data than L3 can hold, L3 cache hit rate collapses to near zero during the load burst, so the frame-drop bottleneck is dominated by DRAM/storage-to-DRAM streaming bandwidth, not by cache-hierarchy latency.

3. Proposed Memory Hierarchy Enhancements (Solution Design)

- Adopt a high-bandwidth unified GDDR6/GDDR6X memory pool (as used in current-generation consoles) so both CPU and GPU draw from a single wide-bandwidth channel instead of contending across separate, narrower pools.
- Use a dedicated hardware decompression/streaming block (e.g., a fast NVMe SSD with a hardware I/O decompression engine) so compressed assets move from storage into DRAM at multi-GB/s without burdening the CPU, directly targeting the scene-load bottleneck.

- Increase L3/last-level cache size and add prefetch hints from the game engine (streaming manager signals upcoming asset needs) so newly streamed data lands in cache just before it's needed, smoothing the transition rather than causing a stall.
- Partition memory bandwidth with QoS so background asset streaming cannot starve the frame currently being rendered, keeping frame pacing steady even during a load burst.

4. Use of Tools

- GPU profilers (e.g., PIX, RenderDoc) — to capture frame-time spikes and correlate them with memory-bandwidth utilization during scene loads.
- DRAMSim2 / Ramulator — to model GDDR6 channel bandwidth under burst asset-streaming traffic.
- Platform SDK memory/IO profilers — to measure real SSD-to-DRAM streaming throughput on the target console.

5. Analysis / Expected Outcome

Because the bottleneck is bandwidth-bound rather than latency-bound, the highest-impact fixes are widening the effective DRAM bandwidth available for streaming and offloading decompression to dedicated hardware, supported by cache prefetching to reduce the residual latency once data lands in DRAM. Together these reduce the duration and severity of the bandwidth spike that causes frame drops during large scene loads.